

Log Procurement (Python)

Config.txt

```
# Log Procurement Tracker Configuration
# Do not edit manually unless you know what you're doing.

excel_password=
last_file_path=C:/Users/vince/OneDrive/-PythonCoding/LogPro/log_procurement.xlsx
login_required=False
window_geometry=1450x1134+1623+140
```

data_layer.py

```
# data_layer.py
# Handles Excel read/write operations for the Log Procurement Tracker app.

from openpyxl import Workbook, load_workbook as openpyxl_load_workbook
from openpyxl.styles import Font, PatternFill, Alignment
from openpyxl.utils import get_column_letter
import os

SHEET_NAMES = {
    "SUPPLIERS": "Suppliers",
    "SUPPLIER_CONTACTS": "SupplierContacts",
    "PROCUREMENTS": "Procurements",
    "PROCUREMENT_GRADES": "ProcurementGrades",
    "PRICE_HISTORY": "PriceHistory",
    "COSTINGS": "Costings",
    "RESALES": "Resales",
    "USERS": "Users",
    "SETTINGS": "Settings",
    "SPECIES_DEFINITIONS": "SpeciesDefinitions",
}

HEADERS = {
    "Suppliers": [
        "SupplierID", "SupplierName", "Address", "ABN", "PaymentTerms", "Notes",
        "CreatedBy", "CreatedDate", "ChangedBy", "ChangedDate"
    ],
    "SupplierContacts": [
        "ContactID", "SupplierID", "ContactName", "Role", "PhoneNumber",
        "MobileNumber", "Email", "Notes", "IsPrimary"
    ],
    "Procurements": [
        "ProcurementID", "ProcurementRef", "SupplierID", "ContactID",
        "AgreementType", "AgreementDetail", "Plantation", "Species",
        "HarvestPeriodStart", "HarvestPeriodEnd", "StartDate", "EndDate",
    ]
}
```

```

        "Status", "AcceptanceDate", "AcceptanceTime", "AcceptanceMethod",
        "AcceptedByPerson", "AcceptanceNotes", "Notes",
        "CreatedBy", "CreatedDate", "ChangedBy", "ChangedDate"
    ],
    "ProcurementGrades": [
        "ProcurementGradeID", "ProcurementRef", "Species", "ProductType", "GradeName",
        "OfferedPricePerTonne", "AgreedPricePerTonne",
        "AgreedTonnes", "DeliveredTonnes", "RemainingTonnes", "Notes"
    ],
    "PriceHistory": [
        "PriceHistoryID", "ProcurementRef", "ProcurementGradeID",
        "ProductType", "GradeName", "PreviousPrice", "NewPrice",
        "ChangeType", "Reason", "EffectiveDateTime", "Notes",
        "RecordedBy", "RecordedDateTime"
    ],
    "Costings": [
        "CostingID", "DestinationCountry", "SellingPriceUSD", "ExchangeRate",
        "ClearanceAUD", "SeaFreightAUD", "TransportAUD", "FumigationAUD",
        "PackingAUD", "ProcurementFeeAUD", "MaxAffordableOfferAUD",
        "MaxAffordableOfferUSD", "RecommendedOfferAUD", "RecommendedOfferUSD",
        "CreatedBy", "CreatedDate"
    ],
    "Resales": [
        "ResaleID", "ProcurementRef", "CustomerName", "SellingPricePerTonne",
        "BuyingPricePerTonne", "Margin", "DateSold", "Notes"
    ],
    "Users": ["UserID", "Username", "PasswordHash", "Role", "IsActive"],
    "Settings": ["SettingKey", "SettingValue"],
    "SpeciesDefinitions": ["SpeciesDefinitionID", "SpeciesName", "IsStandard", "Notes"],
    "GradeDefinitions": ["GradeDefinitionID", "SpeciesName", "ProductType", "GradeName",
    "IsStandard", "Notes"],
}

```

```

STANDARD_GRADES = {
    "Fresh Logs": [
        "SA Grade", "A Grade", "K Grade", "KI Grade",
        "MP Grade", "Big MP", "Pulp"
    ],
    "Burnt Logs": [
        "SA Grade", "A Grade", "K Grade", "KI Grade", "MP Grade"
    ],
}

```

```

def _style_header(ws):
    for cell in ws[1]:
        cell.font = Font(bold=True)
        cell.alignment = Alignment(horizontal="center")
    ws.freeze_panes = "A2"
    ws.auto_filter.ref = ws.dimensions

```

```

def create_new_workbook(filepath, password=None):
    wb = Workbook()

```

```

del wb[wb.sheetnames[0]]
for sheet_name in SHEET_NAMES.values():
    ws = wb.create_sheet(sheet_name)
    ws.append(HEADERS[sheet_name])
    _style_header(ws)

# Store the PDF-derived standard grades in the workbook so they are visible
# and can be extended later without changing Python code.
ws = wb.create_sheet("GradeDefinitions")
ws.append(HEADERS["GradeDefinitions"])
definition_id = 1
for product_type, grades in STANDARD_GRADES.items():
    for grade in grades:
        ws.append([definition_id, "", product_type, grade, True, "Standard grade from Log Procurement
reference PDF"])
        definition_id += 1
    _style_header(ws)

wb.save(filepath)
return wb

def ensure_workbook_structure(filepath):
    if not os.path.exists(filepath):
        return create_new_workbook(filepath)

wb = openpyxl.load_workbook(filepath)
changed = False

required = dict(HEADERS)
required["GradeDefinitions"] = HEADERS["GradeDefinitions"]

incompatible = []
for sheet_name, headers in required.items():
    if sheet_name in wb.sheetnames:
        existing = [wb[sheet_name].cell(1, c).value for c in range(1, len(headers) + 1)]
        if existing[:len(headers)] != headers:
            incompatible.append(sheet_name)

if incompatible:
    # The old fictional Offers/Sales workbook is not migrated. Preserve it
    # as a backup, then create the clean procurement workbook requested by
    # the project. This prevents old columns from being read incorrectly.
    wb.close()
    backup = filepath + ".legacy_backup.xlsx"
    if not os.path.exists(backup):
        os.replace(filepath, backup)
    else:
        os.remove(filepath)
    return create_new_workbook(filepath)

for sheet_name, headers in required.items():
    if sheet_name not in wb.sheetnames:

```

```

        ws = wb.create_sheet(sheet_name)
        ws.append(headers)
        _style_header(ws)
        changed = True

    if changed:
        wb.save(filepath)
    return wb

def load_workbook(filepath, password=None):
    try:
        return openpyxl.load_workbook(filepath)
    except PermissionError:
        raise PermissionError(
            "The Excel file is locked. Close it in Excel and wait for OneDrive to finish syncing."
        )

def save_workbook(wb, filepath, password=None):
    try:
        wb.save(filepath)
    except PermissionError:
        raise PermissionError(
            "The Excel file is locked. Close it in Excel and wait for OneDrive to finish syncing."
        )

def get_next_id(ws, id_column_index=1):
    highest = 0
    for row in ws.iter_rows(min_row=2, values_only=True):
        value = row[id_column_index - 1]
        try:
            if value is not None:
                highest = max(highest, int(value))
        except (ValueError, TypeError):
            pass
    return highest + 1

def get_all_rows(ws, headers):
    records = []
    for row in ws.iter_rows(min_row=2, values_only=True):
        if any(value is not None for value in row):
            records.append(dict(zip(headers, row)))
    return records

def add_row(ws, headers, data):
    new_id = get_next_id(ws, 1)
    values = [new_id] + [data.get(header, "") for header in headers[1:]]
    ws.append(values)
    return new_id

# Suppliers

def get_all_suppliers(wb):

```

```

return get_all_rows(wb[SHEET_NAMES["SUPPLIERS"]], HEADERS["Suppliers"])

def get_supplier_by_id(wb, supplier_id):
    for supplier in get_all_suppliers(wb):
        if str(supplier.get("SupplierID")) == str(supplier_id):
            return supplier
    return None

def add_supplier(wb, data):
    return add_row(wb[SHEET_NAMES["SUPPLIERS"]], HEADERS["Suppliers"], data)

# Supplier contacts

def get_all_supplier_contacts(wb, supplier_id=None):
    records = get_all_rows(wb[SHEET_NAMES["SUPPLIER_CONTACTS"]],
HEADERS["SupplierContacts"])
    if supplier_id is None:
        return records
    return [r for r in records if str(r.get("SupplierID")) == str(supplier_id)]

def add_supplier_contact(wb, data):
    return add_row(wb[SHEET_NAMES["SUPPLIER_CONTACTS"]], HEADERS["SupplierContacts"], data)

# Procurement

def get_all_procurements(wb):
    return get_all_rows(wb[SHEET_NAMES["PROCUREMENTS"]], HEADERS["Procurements"])

def get_procurement_by_ref(wb, procurement_ref):
    for record in get_all_procurements(wb):
        if str(record.get("ProcurementRef", "")) == str(procurement_ref):
            return record
    return None

def get_next_procurement_ref(wb):
    highest = 0
    for record in get_all_procurements(wb):
        ref = str(record.get("ProcurementRef", ""))
        if ref.startswith("PROC-"):
            try:
                highest = max(highest, int(ref[5:]))
            except ValueError:
                pass
    return f"PROC-({highest + 1:04d})"

def add_procurement(wb, data):
    data = dict(data)
    ref = get_next_procurement_ref(wb)
    data["ProcurementRef"] = ref
    procurement_id = add_row(
        wb[SHEET_NAMES["PROCUREMENTS"]],
        HEADERS["Procurements"],

```

```

    data,
)
return procurement_id, ref

def update_procurement(wb, procurement_ref, data):
    ws = wb[SHEET_NAMES["PROCUREMENTS"]]
    headers = HEADERS["Procurements"]
    target_row = None

    for row_number in range(2, ws.max_row + 1):
        if str(ws.cell(row_number, 2).value) == str(procurement_ref):
            target_row = row_number
            break

    if target_row is None:
        raise ValueError(f"Procurement {procurement_ref} was not found.")

    for column_number, header in enumerate(headers, start=1):
        if header in data and header not in ("ProcurementID", "ProcurementRef"):
            ws.cell(target_row, column_number, data[header])

def get_all_procurement_grades(wb, procurement_ref=None):
    records = get_all_rows(wb[SHEET_NAMES["PROCUREMENT_GRADES"]],
HEADERS["ProcurementGrades"])
    if procurement_ref is None:
        return records
    return [r for r in records if str(r.get("ProcurementRef")) == str(procurement_ref)]

def get_procurement_grade_by_id(wb, grade_id):
    for record in get_all_procurement_grades(wb):
        if str(record.get("ProcurementGradeID")) == str(grade_id):
            return record
    return None

def add_procurement_grade(wb, data):
    data = dict(data)
    agreed = float(data.get("AgreedTonnes") or 0)
    delivered = float(data.get("DeliveredTonnes") or 0)
    data["RemainingTonnes"] = max(agreed - delivered, 0)
    return add_row(
        wb[SHEET_NAMES["PROCUREMENT_GRADES"]],
        HEADERS["ProcurementGrades"],
        data,
    )

def update_procurement_grade(wb, grade_id, data):
    ws = wb[SHEET_NAMES["PROCUREMENT_GRADES"]]
    headers = HEADERS["ProcurementGrades"]
    target_row = None

    for row_number in range(2, ws.max_row + 1):
        if str(ws.cell(row_number, 1).value) == str(grade_id):

```

```

    target_row = row_number
    break

if target_row is None:
    raise ValueError(f"Procurement grade {grade_id} was not found.")

data = dict(data)
agreed = float(data.get("AgreedTonnes") or 0)
delivered = float(data.get("DeliveredTonnes") or 0)
data["RemainingTonnes"] = max(agreed - delivered, 0)

for column_number, header in enumerate(headers, start=1):
    if header in data and header != "ProcurementGradeID":
        ws.cell(target_row, column_number, data[header])

def get_species_definitions(wb):
    ws = wb[SHEET_NAMES["SPECIES_DEFINITIONS"]]
    return get_all_rows(ws, HEADERS["SpeciesDefinitions"])

def add_species_definition(wb, species_name, notes="User-added species"):
    species_name = str(species_name).strip()
    if not species_name:
        raise ValueError("Species name is required.")
    for record in get_species_definitions(wb):
        if str(record.get("SpeciesName", "")).strip().lower() == species_name.lower():
            return record.get("SpeciesDefinitionID")
    ws = wb[SHEET_NAMES["SPECIES_DEFINITIONS"]]
    new_id = get_next_id(ws, 1)
    ws.append([new_id, species_name, False, notes])

# A new species receives the standard PDF grades for both product types.
grade_ws = wb["GradeDefinitions"]
next_grade_id = get_next_id(grade_ws, 1)
for product_type, grades in STANDARD_GRADES.items():
    for grade in grades:
        grade_ws.append([next_grade_id, species_name, product_type, grade, True, "Standard grade
from Log Procurement reference PDF"])
        next_grade_id += 1
    return new_id

def get_grade_definitions(wb, species_name=None, product_type=None):
    ws = wb["GradeDefinitions"]
    records = get_all_rows(ws, HEADERS["GradeDefinitions"])
    if species_name:
        exact = [r for r in records if str(r.get("SpeciesName", "")).strip().lower() ==
str(species_name).strip().lower()]
        if exact:
            records = exact
        else:
            records = [r for r in records if not str(r.get("SpeciesName", "")).strip()]
    if product_type:
        records = [r for r in records if r.get("ProductType") == product_type]

```

```
return records
```

```
def add_grade_definition(wb, species_name, product_type, grade_name, notes="User-added grade"):
    species_name = str(species_name).strip()
    product_type = str(product_type).strip()
    grade_name = str(grade_name).strip()
    if not species_name:
        raise ValueError("Species name is required.")
    if product_type not in PRODUCT_TYPES:
        raise ValueError("Product type must be Fresh Logs or Burnt Logs.")
    if not grade_name:
        raise ValueError("Grade name is required.")
    for record in get_grade_definitions(wb, species_name, product_type):
        if str(record.get("GradeName", "")).strip().lower() == grade_name.lower():
            return record.get("GradeDefinitionID")
    ws = wb["GradeDefinitions"]
    new_id = get_next_id(ws, 1)
    ws.append([new_id, species_name, product_type, grade_name, False, notes])
    return new_id
```

```
def update_grade_definition(wb, grade_definition_id, data):
    """Update a grade definition without changing its ID."""
    ws = wb["GradeDefinitions"]
    headers = HEADERS["GradeDefinitions"]
    target_row = None
    for row_number in range(2, ws.max_row + 1):
        if str(ws.cell(row_number, 1).value) == str(grade_definition_id):
            target_row = row_number
            break
    if target_row is None:
        raise ValueError(f"Grade definition {grade_definition_id} was not found.")

    for column_number, header in enumerate(headers, start=1):
        if header in data and header != "GradeDefinitionID":
            value = data[header]
            if header == "GradeName":
                value = str(value).strip()
            if not value:
                raise ValueError("Grade name is required.")
            ws.cell(target_row, column_number, value)
```

```
def delete_grade_definition(wb, grade_definition_id):
    """Delete a grade definition."""
    ws = wb["GradeDefinitions"]
    target_row = None
    for row_number in range(2, ws.max_row + 1):
        if str(ws.cell(row_number, 1).value) == str(grade_definition_id):
            target_row = row_number
            break
    if target_row is None:
        raise ValueError(f"Grade definition {grade_definition_id} was not found.")
    ws.delete_rows(target_row, 1)
```

```

# Price history

def get_all_price_history(wb):
    return get_all_rows(wb[SHEET_NAMES["PRICE_HISTORY"]], HEADERS["PriceHistory"])

def add_price_history(wb, data):
    return add_row(
        wb[SHEET_NAMES["PRICE_HISTORY"]],
        HEADERS["PriceHistory"],
        data,
    )

# Costings and other sheets can be implemented in later chapters.

```

Gui_layer.py

```

# gui_layer.py
# Tkinter GUI for the Log Procurement Tracker app.

import tkinter as tk
from tkinter import ttk, filedialog, messagebox, simpledialog
from datetime import datetime
import os

from data_layer import (
    create_new_workbook, ensure_workbook_structure, load_workbook, save_workbook,
    get_all_suppliers, add_supplier, get_all_supplier_contacts, add_supplier_contact,
    get_all_procurements, get_procurement_by_ref, add_procurement,
    get_all_procurement_grades, add_procurement_grade,
    get_grade_definitions, add_grade_definition, update_grade_definition, delete_grade_definition,
    get_species_definitions, add_species_definition,
    get_all_price_history,
)
from utils import set_setting, get_setting

STATUSES = [
    "Draft", "Waiting for Acceptance", "Accepted",
    "Active", "Completed", "Cancelled"
]

PRODUCT_TYPES = ["Fresh Logs", "Burnt Logs"]
AGREEMENT_TYPES = ["Block", "Coupe", "Harvest", "Contract Number"]

class AddContactDialog:
    def __init__(self, parent):
        self.result = None
        self.dialog = tk.Toplevel(parent)
        self.dialog.title("Add Contact")

```

```

self.dialog.geometry("560x540")
self.dialog.minsize(520, 500)
self.dialog.resizable(True, True)
self.dialog.transient(parent)
self.dialog.grab_set()

outer = ttk.Frame(self.dialog, padding=20)
outer.pack(fill="both", expand=True)

ttk.Label(outer, text="Supplier Contact",
          font=("Segoe UI", 14, "bold")).pack(anchor="w", pady=(0, 12))

content = ttk.Frame(outer)
content.pack(fill="both", expand=True)

fields = [
    ("Contact Name *", "ContactName"),
    ("Role / Position", "Role"),
    ("Phone Number", "PhoneNumber"),
    ("Mobile Number", "MobileNumber"),
    ("Email", "Email"),
    ("Notes", "Notes"),
]
self.vars = {}
for label, key in fields:
    ttk.Label(content, text=label).pack(anchor="w", pady=(7, 2))
    var = tk.StringVar()
    self.vars[key] = var
    ttk.Entry(content, textvariable=var).pack(fill="x", pady=(0, 3))

self.primary_var = tk.BooleanVar(value=False)
ttk.Checkbutton(content, text="Primary contact", variable=self.primary_var).pack(anchor="w",
pady=10)

buttons = ttk.Frame(outer)
buttons.pack(fill="x", side="bottom", pady=(12, 0))
ttk.Button(buttons, text="Save Contact", command=self._save, width=20).pack(side="left",
padx=5)
ttk.Button(buttons, text="Cancel", command=self.dialog.destroy, width=20).pack(side="left",
padx=5)

self.dialog.protocol("WM_DELETE_WINDOW", self.dialog.destroy)
self.vars["ContactName"].set("")
self.dialog.update_idletasks()
self.dialog.focus_force()
parent.wait_window(self.dialog)

def _save(self):
    if not self.vars["ContactName"].get().strip():
        messagebox.showerror("Validation Error", "Contact Name is required.", parent=self.dialog)
        return
    self.result = {key: var.get().strip() for key, var in self.vars.items()}

```

```
self.result["IsPrimary"] = self.primary_var.get()
self.dialog.destroy()
```

```
class AddSupplierDialog:
```

```
    """Supplier form with an expandable list of supplier contacts."""
```

```
    def __init__(self, parent, excel_filepath):
```

```
        self.result = False
```

```
        self.result_supplier_id = None
```

```
        self.result_supplier_name = ""
```

```
        self.excel_filepath = excel_filepath
```

```
        self._contact_data = {}
```

```
        self.dialog = tk.Toplevel(parent)
```

```
        self.dialog.title("Add Supplier")
```

```
        screen_h = self.dialog.winfo_screenheight()
```

```
        dialog_h = min(760, max(600, screen_h - 90))
```

```
        self.dialog.geometry(f"760x{dialog_h}")
```

```
        self.dialog.minsize(680, 600)
```

```
        self.dialog.resizable(True, True)
```

```
        self.dialog.transient(parent)
```

```
        self.dialog.grab_set()
```

```
        outer = ttk.Frame(self.dialog, padding=18)
```

```
        outer.pack(fill="both", expand=True)
```

```
        outer.rowconfigure(1, weight=1)
```

```
        outer.columnconfigure(0, weight=1)
```

```
        ttk.Label(outer, text="Add New Supplier",
```

```
                  font=("Segoe UI", 15, "bold")).grid(row=0, column=0, sticky="w", pady=(0, 12))
```

```
        body = ttk.Frame(outer)
```

```
        body.grid(row=1, column=0, sticky="nsew")
```

```
        body.rowconfigure(1, weight=1)
```

```
        body.columnconfigure(0, weight=1)
```

```
        supplier_frame = ttk.LabelFrame(body, text="Supplier Details", padding=12)
```

```
        supplier_frame.grid(row=0, column=0, sticky="ew")
```

```
        supplier_frame.columnconfigure(1, weight=1)
```

```
        fields = [
```

```
            ("Supplier Name *", "SupplierName"),
```

```
            ("Address", "Address"),
```

```
            ("ABN", "ABN"),
```

```
            ("Payment Terms", "PaymentTerms"),
```

```
            ("Notes", "Notes"),
```

```
        ]
```

```
        self.vars = {}
```

```
        for i, (label, key) in enumerate(fields):
```

```
            ttk.Label(supplier_frame, text=label).grid(row=i, column=0, sticky="w", padx=5, pady=5)
```

```
            var = tk.StringVar()
```

```

self.vars[key] = var
ttk.Entry(supplier_frame, textvariable=var).grid(
    row=i, column=1, sticky="ew", padx=5, pady=5
)

contacts_frame = ttk.LabelFrame(body, text="Supplier Contacts", padding=10)
contacts_frame.grid(row=1, column=0, sticky="nsew", pady=(12, 0))
contacts_frame.rowconfigure(1, weight=1)
contacts_frame.columnconfigure(0, weight=1)

ttk.Label(
    contacts_frame,
    text="Add as many contacts as needed. Each contact can have its own phone, mobile and
email.",
).grid(row=0, column=0, sticky="w", pady=(0, 8))

tree_area = ttk.Frame(contacts_frame)
tree_area.grid(row=1, column=0, sticky="nsew")
tree_area.rowconfigure(0, weight=1)
tree_area.columnconfigure(0, weight=1)

columns = ("Name", "Role", "Phone", "Mobile", "Email", "Primary")
self.contacts_tree = ttk.Treeview(tree_area, columns=columns, show="headings")
widths = {"Name": 150, "Role": 120, "Phone": 110, "Mobile": 110, "Email": 220, "Primary": 70}
for col in columns:
    self.contacts_tree.heading(col, text=col)
    self.contacts_tree.column(col, width=widths[col], minwidth=60)
self.contacts_tree.grid(row=0, column=0, sticky="nsew")

contact_scroll = ttk.Scrollbar(tree_area, orient="vertical", command=self.contacts_tree.yview)
contact_scroll.grid(row=0, column=1, sticky="ns")
self.contacts_tree.configure(yscrollcommand=contact_scroll.set)

contact_buttons = ttk.Frame(contacts_frame)
contact_buttons.grid(row=2, column=0, sticky="ew", pady=(8, 0))
ttk.Button(contact_buttons, text="Add Contact",
    command=self._add_contact, width=18).pack(side="left", padx=4)
ttk.Button(contact_buttons, text="Remove Selected",
    command=self._remove_contact, width=18).pack(side="left", padx=4)

bottom = ttk.Frame(outer)
bottom.grid(row=2, column=0, sticky="ew", pady=(14, 0))
ttk.Button(bottom, text="Save Supplier",
    command=self._save_supplier, width=22).pack(side="left", padx=5)
ttk.Button(bottom, text="Cancel",
    command=self.dialog.destroy, width=22).pack(side="left", padx=5)

self.dialog.protocol("WM_DELETE_WINDOW", self.dialog.destroy)
self.vars["SupplierName"].set("")
self.dialog.update_idletasks()
self.dialog.focus_force()
parent.wait_window(self.dialog)

```

```

def _add_contact(self):
    dialog = AddContactDialog(self.dialog)
    if dialog.result:
        c = dialog.result
        item = self.contacts_tree.insert("", "end", values=(
            c["ContactName"], c["Role"], c["PhoneNumber"],
            c["MobileNumber"], c["Email"], "Yes" if c["IsPrimary"] else ""
        ))
        self._contact_data[item] = c

def _remove_contact(self):
    for item in self.contacts_tree.selection():
        self._contact_data.pop(item, None)
        self.contacts_tree.delete(item)

def _save_supplier(self):
    supplier_data = {key: var.get().strip() for key, var in self.vars.items()}
    if not supplier_data["SupplierName"]:
        messagebox.showerror("Validation Error", "Supplier Name is required.", parent=self.dialog)
        return

    try:
        wb = load_workbook(self.excel_filepath)
        supplier_id = add_supplier(wb, supplier_data)

        contacts = []
        for item in self.contacts_tree.get_children():
            c = self._contact_data.get(item)
            if c:
                c = dict(c)
                c["SupplierID"] = supplier_id
                contacts.append(c)

        primary_seen = False
        for c in contacts:
            if c.get("IsPrimary"):
                if primary_seen:
                    c["IsPrimary"] = False
                else:
                    primary_seen = True
        add_supplier_contact(wb, c)

        save_workbook(wb, self.excel_filepath)
        wb.close()
        self.result = True
        self.result_supplier_id = supplier_id
        self.result_supplier_name = supplier_data["SupplierName"]
        self.dialog.destroy()
    except Exception as error:
        messagebox.showerror("Save Error", f"Could not save the supplier:\n{error}", parent=self.dialog)

```

```
class ManageSupplierDialog:
```

```
    """Open an existing supplier and add contacts without creating a duplicate supplier."""
```

```
    def __init__(self, parent, excel_filepath, supplier_id):
```

```
        self.result = False
```

```
        self.excel_filepath = excel_filepath
```

```
        self.supplier_id = supplier_id
```

```
        self.dialog = tk.Toplevel(parent)
```

```
        self.dialog.title("Supplier Details")
```

```
        self.dialog.geometry("760x700")
```

```
        self.dialog.minsize(680, 600)
```

```
        self.dialog.resizable(True, True)
```

```
        self.dialog.transient(parent)
```

```
        self.dialog.grab_set()
```

```
        outer = ttk.Frame(self.dialog, padding=18)
```

```
        outer.pack(fill="both", expand=True)
```

```
        outer.rowconfigure(1, weight=1)
```

```
        outer.columnconfigure(0, weight=1)
```

```
        self.title_label = ttk.Label(outer, text="Supplier Details",
```

```
                                     font=("Segoe UI", 15, "bold"))
```

```
        self.title_label.grid(row=0, column=0, sticky="w", pady=(0, 12))
```

```
        body = ttk.Frame(outer)
```

```
        body.grid(row=1, column=0, sticky="nsew")
```

```
        body.rowconfigure(1, weight=1)
```

```
        body.columnconfigure(0, weight=1)
```

```
        self.supplier_info = ttk.LabelFrame(body, text="Supplier Details", padding=12)
```

```
        self.supplier_info.grid(row=0, column=0, sticky="ew")
```

```
        self.supplier_info.columnconfigure(1, weight=1)
```

```
        self.info_vars = {}
```

```
        for row, (label, key) in enumerate([
```

```
            ("Supplier Name", "SupplierName"),
```

```
            ("Address", "Address"),
```

```
            ("ABN", "ABN"),
```

```
            ("Payment Terms", "PaymentTerms"),
```

```
        ]):
```

```
            ttk.Label(self.supplier_info, text=label).grid(row=row, column=0, sticky="w", padx=5, pady=4)
```

```
            var = tk.StringVar()
```

```
            self.info_vars[key] = var
```

```
            ttk.Entry(self.supplier_info, textvariable=var, state="readonly").grid(
```

```
                row=row, column=1, sticky="ew", padx=5, pady=4
```

```
            )
```

```
        contacts = ttk.LabelFrame(body, text="Contacts", padding=10)
```

```
        contacts.grid(row=1, column=0, sticky="nsew", pady=(12, 0))
```

```
        contacts.rowconfigure(0, weight=1)
```

```

contacts.columnconfigure(0, weight=1)

columns = ("Name", "Role", "Phone", "Mobile", "Email", "Primary")
self.contacts_tree = ttk.Treeview(contacts, columns=columns, show="headings")
widths = {"Name":150, "Role":120, "Phone":110, "Mobile":110, "Email":220, "Primary":70}
for col in columns:
    self.contacts_tree.heading(col, text=col)
    self.contacts_tree.column(col, width=widths[col], minwidth=60)
self.contacts_tree.grid(row=0, column=0, sticky="nsew")
scroll = ttk.Scrollbar(contacts, orient="vertical", command=self.contacts_tree.yview)
scroll.grid(row=0, column=1, sticky="ns")
self.contacts_tree.configure(yscrollcommand=scroll.set)

contact_buttons = ttk.Frame(contacts)
contact_buttons.grid(row=1, column=0, sticky="ew", pady=(8, 0))
ttk.Button(contact_buttons, text="Add Contact", width=18,
            command=self._add_contact).pack(side="left", padx=4)
ttk.Button(contact_buttons, text="Remove Selected", width=18,
            command=self._remove_contact).pack(side="left", padx=4)

bottom = ttk.Frame(outer)
bottom.grid(row=2, column=0, sticky="ew", pady=(14, 0))
ttk.Button(bottom, text="Close", width=20,
            command=self.dialog.destroy).pack(side="right", padx=5)

self._load_supplier()
self.dialog.protocol("WM_DELETE_WINDOW", self.dialog.destroy)
parent.wait_window(self.dialog)

def _load_supplier(self):
    try:
        wb = load_workbook(self.excel_filepath)
        suppliers = get_all_suppliers(wb)
        contacts = get_all_supplier_contacts(wb, self.supplier_id)
        wb.close()
        supplier = next((s for s in suppliers if str(s.get("SupplierID")) == str(self.supplier_id)), None)
        if not supplier:
            raise ValueError("Supplier could not be found.")
        self.title_label.configure(text=f"Supplier: {supplier.get('SupplierName', '')}")
        for key, var in self.info_vars.items():
            var.set(str(supplier.get(key, "")) or "")
        self._display_contacts(contacts)
    except Exception as error:
        messagebox.showerror("Load Error", str(error), parent=self.dialog)

def _display_contacts(self, contacts):
    for item in self.contacts_tree.get_children():
        self.contacts_tree.delete(item)
    for c in contacts:
        self.contacts_tree.insert("", "end", values=(
            c.get("ContactName", ""), c.get("Role", ""),
            c.get("PhoneNumber", ""), c.get("MobileNumber", "")),

```

```
        c.get("Email", ""), "Yes" if c.get("IsPrimary") else ""
    ))
```

```
def _add_contact(self):
```

```
    dialog = AddContactDialog(self.dialog)
```

```
    if not dialog.result:
```

```
        return
```

```
    try:
```

```
        wb = load_workbook(self.excel_filepath)
```

```
        data = dict(dialog.result)
```

```
        data["SupplierID"] = self.supplier_id
```

```
        add_supplier_contact(wb, data)
```

```
        save_workbook(wb, self.excel_filepath)
```

```
        wb.close()
```

```
        self._load_supplier()
```

```
        self.result = True
```

```
    except Exception as error:
```

```
        messagebox.showerror("Save Error", f"Could not save the contact:\n{error}", parent=self.dialog)
```

```
def _remove_contact(self):
```

```
    selection = self.contacts_tree.selection()
```

```
    if not selection:
```

```
        messagebox.showwarning("Select Contact", "Select a contact to remove.", parent=self.dialog)
```

```
        return
```

```
    messagebox.showinfo(
```

```
        "Contact Removal",
```

```
        "Contact deletion will be added in the validation/editing chapter. For now, use Add Contact to
```

```
add another contact.",
```

```
        parent=self.dialog,
```

```
    )
```

```
class FirstRunDialog:
```

```
    """First-run dialog required by main.py."""
```

```
def __init__(self):
```

```
    self.result_path = None
```

```
    self.root = tk.Tk()
```

```
    self.root.title("Log Procurement Tracker - First Run")
```

```
    self.root.geometry("560x280")
```

```
    self.root.resizable(False, False)
```

```
    frame = ttk.Frame(self.root, padding=25)
```

```
    frame.pack(fill="both", expand=True)
```

```
    ttk.Label(
```

```
        frame,
```

```
        text="Welcome to Log Procurement Tracker",
```

```
        font=("Segoe UI", 16, "bold")
```

```
    ).pack(pady=(10, 10))
```

```
    ttk.Label(
```

```

        frame,
        text="Choose the Excel workbook for LogPro.",
        justify="center"
    ).pack(pady=10)

    buttons = ttk.Frame(frame)
    buttons.pack(pady=15)

    ttk.Button(
        buttons, text="Create New File", width=22,
        command=self._on_create_new
    ).pack(side="left", padx=8)

    ttk.Button(
        buttons, text="Open Existing File", width=22,
        command=self._on_open_existing
    ).pack(side="left", padx=8)

    self.root.protocol("WM_DELETE_WINDOW", self._on_cancel)
    self.root.mainloop()

def _on_create_new(self):
    filepath = filedialog.asksaveasfilename(
        title="Save New Excel File",
        defaultextension=".xlsx",
        filetypes=[("Excel files", "*.xlsx")],
        initialfile="log_procurement.xlsx",
    )
    if not filepath:
        return

    try:
        workbook = create_new_workbook(filepath)
        workbook.close()
        set_setting("last_file_path", filepath)
        self.result_path = filepath
        self.root.destroy()
    except Exception as error:
        messagebox.showerror("Error", f"Failed to create file:\n{error}", parent=self.root)

def _on_open_existing(self):
    filepath = filedialog.askopenfilename(
        title="Select Existing File",
        filetypes=[("Excel files", "*.xlsx")],
        initialdir=os.path.expanduser("~"),
    )
    if not filepath:
        return

    try:
        workbook = ensure_workbook_structure(filepath)
        workbook.close()

```

```

        set_setting("last_file_path", filepath)
        self.result_path = filepath
        self.root.destroy()
    except Exception as error:
        messagebox.showerror("Error", f"Failed to open file:\n{error}", parent=self.root)

def _on_cancel(self):
    self.root.destroy()

class MainWindow:
    def __init__(self, root, excel_filepath):
        self.root = root
        self.excel_filepath = excel_filepath
        self.current_procurement_ref = None
        self.supplier_id_map = {}
        self.contact_id_map = {}
        self.root.title("Log Procurement Tracker")
        self.root.minsize(1100, 700)

        # Restore the user's last window size and position.
        # If no saved geometry exists, use the normal first-run size.
        saved_geometry = get_setting("window_geometry", "")
        if saved_geometry:
            try:
                self.root.geometry(saved_geometry)
            except tk.TclError:
                self.root.geometry("1450x920")
        else:
            self.root.geometry("1450x920")

        self.root.protocol("WM_DELETE_WINDOW", self._on_close)

        try:
            wb = ensure_workbook_structure(self.excel_filepath)
            wb.close()
        except Exception as error:
            messagebox.showerror("Workbook Error", str(error), parent=self.root)

        self.tab_control = ttk.Notebook(root)
        self.tabs = {
            "Home": ttk.Frame(self.tab_control),
            "Procurements": ttk.Frame(self.tab_control),
            "Price History": ttk.Frame(self.tab_control),
            "Suppliers": ttk.Frame(self.tab_control),
            "Costing": ttk.Frame(self.tab_control),
            "Reports": ttk.Frame(self.tab_control),
            "Resales": ttk.Frame(self.tab_control),
            "Settings": ttk.Frame(self.tab_control),
        }
        for name, frame in self.tabs.items():
            self.tab_control.add(frame, text=name)

```

```

self.tab_control.pack(fill="both", expand=True)

self._init_home_tab()
self._init_procurements_tab()
self._init_price_history_tab()
self._init_suppliers_tab()
self._init_placeholder_tabs()

self._refresh_supplier_choices()
self._refresh_procurement_table()
self._refresh_price_history()
self._refresh_suppliers()

def _init_home_tab(self):
    frame = ttk.Frame(self.tabs["Home"], padding=30)
    frame.pack(fill="both", expand=True)
    ttk.Label(frame, text="Log Procurement Tracker",
               font=("Segoe UI", 20, "bold")).pack(pady=(30, 10))
    self.home_summary = ttk.Label(frame, text="", font=("Segoe UI", 12), justify="left")
    self.home_summary.pack(pady=20)
    self._refresh_home()

def _refresh_home(self):
    try:
        wb = load_workbook(self.excel_filepath)
        records = get_all_procurements(wb)
        grades = get_all_procurement_grades(wb)
        wb.close()
        counts = {status: 0 for status in STATUSES}
        for r in records:
            if r.get("Status") in counts:
                counts[r.get("Status")] += 1
        agreed = sum(float(g.get("AgreedTonnes")) or 0 for g in grades)
        delivered = sum(float(g.get("DeliveredTonnes")) or 0 for g in grades)
        remaining = sum(float(g.get("RemainingTonnes")) or 0 for g in grades)
        self.home_summary.config(text=(
            f"Total procurements: {len(records)}\n\n"
            f"Draft: {counts['Draft']}\n\n"
            f"Waiting for Acceptance: {counts['Waiting for Acceptance']}\n\n"
            f"Accepted: {counts['Accepted']}\n\n"
            f"Active: {counts['Active']}\n\n"
            f"Completed: {counts['Completed']}\n\n"
            f"Cancelled: {counts['Cancelled']}\n\n"
            f"Agreed tonnes: {agreed:,.2f}\n\n"
            f"Delivered tonnes: {delivered:,.2f}\n\n"
            f"Remaining tonnes: {remaining:,.2f}"
        ))
    except Exception as error:
        self.home_summary.config(text=f"Could not load dashboard:\n{error}")

def _init_procurements_tab(self):
    tab = self.tabs["Procurements"]

```

```

outer = ttk.Frame(tab, padding=(8, 8, 8, 8))
outer.pack(fill="both", expand=True)

header = ttk.Frame(outer)
header.pack(fill="x", pady=(0, 8))
ttk.Label(
    header, text="Procurement Agreement",
    font=("Segoe UI", 16, "bold")
).pack(side="left")

# The register lives to the right of the agreement screen.
# Only a narrow strip is visible until the user clicks the arrow.
slide_area = ttk.Frame(outer)
slide_area.pack(fill="both", expand=True)

self.proc_register_visible = False
self.proc_slide_animating = False
self.proc_slide_x = 0
self.proc_peek_width = 38
self.proc_page_width = 0

self.proc_slide_canvas = tk.Canvas(
    slide_area, highlightthickness=0, borderwidth=0
)
self.proc_slide_canvas.pack(fill="both", expand=True)

self.proc_agreement_page = ttk.Frame(self.proc_slide_canvas)
self.proc_register_page = ttk.Frame(self.proc_slide_canvas)

self.proc_agreement_window = self.proc_slide_canvas.create_window(
    0, 0, window=self.proc_agreement_page, anchor="nw"
)
self.proc_register_window = self.proc_slide_canvas.create_window(
    0, 0, window=self.proc_register_page, anchor="nw"
)

self.proc_arrow_button = ttk.Button(
    slide_area,
    text="←",
    width=3,
    command=self._toggle_procurement_register
)
self.proc_arrow_button.place(relx=1.0, x=-3, rely=0.5, anchor="e")

self.proc_slide_canvas.bind("<Configure>", self._layout_procurement_pages)

# ----- Procurement Agreement page -----
form_canvas = tk.Canvas(self.proc_agreement_page, highlightthickness=0)
form_scrollbar = ttk.Scrollbar(
    self.proc_agreement_page, orient="vertical", command=form_canvas.yview
)
form = ttk.Frame(form_canvas, padding=5)

```

```

form.bind(
    "<Configure>",
    lambda e: form_canvas.configure(scrollregion=form_canvas.bbox("all"))
)
form_canvas.create_window((0, 0), window=form, anchor="nw")
form_canvas.configure(yscrollcommand=form_scrollbar.set)
form_canvas.pack(side="left", fill="both", expand=True)
form_scrollbar.pack(side="right", fill="y")

self.proc_vars = {}

supplier_frame = ttk.LabelFrame(form, text="1. Supplier & Contact", padding=12)
supplier_frame.pack(fill="x", pady=(0, 10))
supplier_frame.columnconfigure(1, weight=1)
supplier_frame.columnconfigure(3, weight=1)

self.supplier_combo = self._add_combo_field(
    supplier_frame, "Supplier", "SupplierID", 0, 0,
    self._on_supplier_changed
)
self.contact_combo = self._add_combo_field(
    supplier_frame, "Contact", "ContactID", 0, 2,
    self._on_contact_changed
)
self._add_readonly_field(supplier_frame, "Supplier Address", "SupplierAddress", 1, 0)
self._add_readonly_field(supplier_frame, "ABN", "SupplierABN", 1, 2)
self._add_readonly_field(supplier_frame, "Payment Terms", "SupplierPaymentTerms", 2, 0)
self._add_readonly_field(supplier_frame, "Contact Role", "ContactRole", 2, 2)
self._add_readonly_field(supplier_frame, "Contact Phone", "ContactPhone", 3, 0)
self._add_readonly_field(supplier_frame, "Contact Mobile", "ContactMobile", 3, 2)
self._add_readonly_field(supplier_frame, "Contact Email", "ContactEmail", 4, 0)

agreement_frame = ttk.LabelFrame(form, text="2. Agreement", padding=12)
agreement_frame.pack(fill="x", pady=(0, 10))
agreement_frame.columnconfigure(1, weight=1)
agreement_frame.columnconfigure(3, weight=2)

ttk.Label(agreement_frame, text="Agreement Type").grid(
    row=0, column=0, sticky="w", padx=5, pady=5
)
self.proc_vars["AgreementType"] = tk.StringVar()
agreement_combo = ttk.Combobox(
    agreement_frame,
    textvariable=self.proc_vars["AgreementType"],
    values=AGREEMENT_TYPES,
    state="readonly"
)
agreement_combo.grid(row=0, column=1, sticky="ew", padx=5, pady=5)
agreement_combo.bind("<<ComboboxSelected>>", self._on_agreement_type_changed)

self.agreement_detail_label = ttk.Label(agreement_frame, text="Details")
self.agreement_detail_label.grid(

```

```

        row=0, column=2, sticky="w", padx=5, pady=5
    )
    self.proc_vars["AgreementDetail"] = tk.StringVar()
    self.agreement_detail_entry = ttk.Entry(
        agreement_frame, textvariable=self.proc_vars["AgreementDetail"]
    )
    self.agreement_detail_entry.grid(
        row=0, column=3, sticky="ew", padx=5, pady=5
    )

    timber_frame = ttk.LabelFrame(form, text="3. Plantation & Harvest", padding=12)
    timber_frame.pack(fill="x", pady=(0, 10))
    for col in range(4):
        timber_frame.columnconfigure(col, weight=1)
    self._add_entry_field(timber_frame, "Plantation", "Plantation", 0, 0)
    self._add_entry_field(timber_frame, "Harvest Period Start", "HarvestPeriodStart", 1, 0)
    self._add_entry_field(timber_frame, "Harvest Period End", "HarvestPeriodEnd", 1, 2)
    self._add_entry_field(timber_frame, "Agreement Start Date", "StartDate", 2, 0)
    self._add_entry_field(timber_frame, "Agreement End Date", "EndDate", 2, 2)

    grade_frame = ttk.LabelFrame(form, text="4. Grades, Products, Prices & Tonnes", padding=12)
    grade_frame.pack(fill="both", expand=True, pady=(0, 10))
    ttk.Label(
        grade_frame,
        text="Add one row for each grade/product combination. Standard grades come from the
supplied PDF; new grades can be added."
    ).pack(anchor="w", pady=(0, 8))

    # Inline grade-entry table. No popup is used.
    self.grade_rows = []
    self.grade_table_frame = tk.Frame(grade_frame, bg="white")
    self.grade_table_frame.pack(fill="both", expand=True, padx=8, pady=8)

    # Keep one dedicated white row at the bottom so the grade list has a clear lower border.
    self.grade_table_bottom_border = tk.Frame(self.grade_table_frame, bg="white", height=16)
    self.grade_table_bottom_border.grid(row=1, column=0, columnspan=8, sticky="ew")

    headers = [
        ("Species", 110), ("Product Type", 120), ("Grade", 125),
        ("Offered $/t AUD", 115), ("Agreed $/t AUD", 115),
        ("Agreed Tonnes", 105), ("Delivered Tonnes", 115),
        ("Remaining Tonnes", 115),
    ]
    for col, (heading, width) in enumerate(headers):
        tk.Label(
            self.grade_table_frame,
            text=heading,
            anchor="center",
            bg="white",
            font=("Segoe UI", 9, "bold")
        ).grid(row=0, column=col, sticky="ew", padx=2, pady=(0, 5))
        self.grade_table_frame.columnconfigure(col, weight=1, minsize=width)

```

```

grade_buttons = ttk.Frame(grade_frame)
grade_buttons.pack(fill="x", pady=(8, 0))

ttk.Button(
    grade_buttons, text="Add Grade",
    command=self._add_grade_row, width=18
).pack(side="left", padx=4)

ttk.Button(
    grade_buttons, text="Remove Grade",
    command=self._remove_grade_row, width=18
).pack(side="left", padx=4)

ttk.Button(
    grade_buttons, text="⚙️",
    command=self._open_grade_settings,
    width=4
).pack(side="right", padx=4)

self._grade_row_settings = {
    "species": "",
    "product_type": "Fresh Logs",
    "grades": [],
}
self._load_grade_settings()
self._add_grade_row(focus_grade=False)

status_frame = ttk.LabelFrame(form, text="5. Status & Acceptance", padding=12)
status_frame.pack(fill="x", pady=(0, 10))
for col in range(4):
    status_frame.columnconfigure(col, weight=1)
self._add_combo_field(status_frame, "Status", "Status", 0, 0, None, STATUSES)
self._add_entry_field(status_frame, "Acceptance Date", "AcceptanceDate", 0, 2)
self._add_entry_field(status_frame, "Acceptance Time", "AcceptanceTime", 1, 0)
self._add_combo_field(
    status_frame, "Acceptance Method", "AcceptanceMethod", 1, 2,
    None, ["Email", "Phone", "In Person", "Contract", "Other"]
)
self._add_entry_field(status_frame, "Accepted By Person", "AcceptedByPerson", 2, 0)

ttk.Label(status_frame, text="Acceptance Notes").grid(
    row=2, column=2, sticky="w", padx=5, pady=5
)
self.proc_acceptance_notes = tk.Text(status_frame, height=3)
self.proc_acceptance_notes.grid(
    row=3, column=0, columnspan=4, sticky="ew", padx=5, pady=5
)
ttk.Label(status_frame, text="General Notes").grid(
    row=4, column=0, sticky="w", padx=5, pady=5
)
self.proc_notes = tk.Text(status_frame, height=3)

```

```

self.proc_notes.grid(
    row=5, column=0, columnspan=4, sticky="ew", padx=5, pady=5
)

buttons = ttk.Frame(form)
buttons.pack(fill="x", pady=8)
ttk.Button(
    buttons, text="New Procurement",
    command=self._clear_procurement_form, width=20
).pack(side="left", padx=5)
ttk.Button(
    buttons, text="Save Procurement",
    command=self._save_procurement, width=20
).pack(side="left", padx=5)
ttk.Button(
    buttons, text="Update Selected",
    command=self._update_procurement, width=20
).pack(side="left", padx=5)
ttk.Button(
    buttons, text="Refresh",
    command=self._refresh_procurement_table, width=15
).pack(side="left", padx=5)

# ----- Procurement Register page -----
register_frame = ttk.LabelFrame(
    self.proc_register_page, text="Procurement Register", padding=8
)
register_frame.pack(fill="both", expand=True)
register_frame.rowconfigure(0, weight=1)
register_frame.columnconfigure(0, weight=1)

columns = (
    "Ref", "Supplier", "Agreement", "Detail", "Plantation", "Species",
    "Agreed", "Delivered", "Remaining", "Status"
)
self.proc_tree = ttk.Treeview(
    register_frame, columns=columns, show="headings"
)
headings = {
    "Ref": "Reference", "Supplier": "Supplier", "Agreement": "Agreement",
    "Detail": "Detail", "Plantation": "Plantation", "Species": "Species",
    "Agreed": "Agreed t", "Delivered": "Delivered t",
    "Remaining": "Remaining t", "Status": "Status"
}
widths = {
    "Ref": 95, "Supplier": 160, "Agreement": 120, "Detail": 150,
    "Plantation": 140, "Species": 100, "Agreed": 90,
    "Delivered": 100, "Remaining": 100, "Status": 150
}
for col in columns:
    self.proc_tree.heading(col, text=headings[col])
    self.proc_tree.column(col, width=widths[col], minwidth=70)

```

```

self.proc_tree.grid(row=0, column=0, sticky="nsew")

reg_vsb = ttk.Scrollbar(
    register_frame, orient="vertical", command=self.proc_tree.yview
)
reg_vsb.grid(row=0, column=1, sticky="ns")
reg_hsb = ttk.Scrollbar(
    register_frame, orient="horizontal", command=self.proc_tree.xview
)
reg_hsb.grid(row=1, column=0, sticky="ew")
self.proc_tree.configure(
    yscrollcommand=reg_vsb.set, xscrollcommand=reg_hsb.set
)
self.proc_tree.bind("<<TreeviewSelect>>", self._on_procurement_selected)

self.root.after(100, self._layout_procurement_pages)

def _layout_procurement_pages(self, event=None):
    canvas = self.proc_slide_canvas
    width = max(canvas.winfo_width(), 700)
    height = max(canvas.winfo_height(), 450)

    # Agreement occupies almost the whole viewport, leaving a small
    # visible register strip on the right.
    agreement_width = max(width - self.proc_peek_width, 500)
    register_width = width
    shift = agreement_width

    self.proc_page_width = agreement_width
    self.proc_slide_max_x = shift

    self.proc_slide_canvas.itemconfigure(
        self.proc_agreement_window, width=agreement_width, height=height
    )
    self.proc_slide_canvas.itemconfigure(
        self.proc_register_window, width=register_width, height=height
    )

    # Register starts immediately after the agreement page.
    # Move both pages as one horizontal strip.
    self.proc_slide_canvas.coords(
        self.proc_agreement_window, -self.proc_slide_x, 0
    )
    self.proc_slide_canvas.coords(
        self.proc_register_window, agreement_width - self.proc_slide_x, 0
    )

    total_width = agreement_width + register_width
    self.proc_slide_canvas.configure(
        scrollregion=(0, 0, total_width, height)
    )

```

```

def _toggle_procurement_register(self):
    if self.proc_slide_animating:
        return
    self.proc_register_visible = not self.proc_register_visible
    self.proc_slide_target = (
        self.proc_slide_max_x if self.proc_register_visible else 0
    )
    self.proc_arrow_button.configure(
        text=">" if self.proc_register_visible else "<"
    )
    self._animate_procurement_slide()

def _animate_procurement_slide(self):
    distance = self.proc_slide_target - self.proc_slide_x
    if abs(distance) <= 2:
        self.proc_slide_x = self.proc_slide_target
        self.proc_slide_animating = False
        self._layout_procurement_pages()
        return

    self.proc_slide_animating = True
    self.proc_slide_x += distance * 0.20
    self._layout_procurement_pages()
    self.root.after(16, self._animate_procurement_slide)

def _add_entry_field(self, parent, label, key, row, col):
    ttk.Label(parent, text=label).grid(row=row, column=col, sticky="w", padx=5, pady=5)
    var = tk.StringVar()
    self.proc_vars[key] = var
    ttk.Entry(parent, textvariable=var, width=30).grid(
        row=row, column=col + 1, sticky="ew", padx=5, pady=5
    )

def _add_combo_field(self, parent, label, key, row, col, callback=None, values=None):
    ttk.Label(parent, text=label).grid(row=row, column=col, sticky="w", padx=5, pady=5)
    var = tk.StringVar()
    self.proc_vars[key] = var
    combo = ttk.Combobox(parent, textvariable=var, values=values or [],
        state="readonly", width=28)
    combo.grid(row=row, column=col + 1, sticky="ew", padx=5, pady=5)
    if callback:
        combo.bind("<<ComboboxSelected>>", callback)
    return combo

def _add_readonly_field(self, parent, label, key, row, col):
    ttk.Label(parent, text=label).grid(row=row, column=col, sticky="w", padx=5, pady=5)
    var = tk.StringVar()
    self.proc_vars[key] = var
    ttk.Entry(parent, textvariable=var, state="readonly", width=30).grid(
        row=row, column=col + 1, sticky="ew", padx=5, pady=5
    )

```

```

def _refresh_supplier_choices(self):
    try:
        wb = load_workbook(self.excel_filepath)
        suppliers = get_all_suppliers(wb)
        wb.close()
        self.supplier_id_map = {}
        values = []
        for s in suppliers:
            display = f"{s.get('SupplierName', '')} (ID: {s.get('SupplierID', '')})"
            values.append(display)
            self.supplier_id_map[display] = s.get("SupplierID")
        if hasattr(self, "supplier_combo"):
            self.supplier_combo["values"] = values
    except Exception as error:
        if hasattr(self, "supplier_combo"):
            self.supplier_combo["values"] = []

def _on_supplier_changed(self, event=None):
    display = self.proc_vars["SupplierID"].get()
    supplier_id = self.supplier_id_map.get(display)
    if not supplier_id:
        return
    try:
        wb = load_workbook(self.excel_filepath)
        supplier = next((s for s in get_all_suppliers(wb)
                        if str(s.get("SupplierID")) == str(supplier_id)), None)
        contacts = get_all_supplier_contacts(wb, supplier_id)
        wb.close()

        if supplier:
            self.proc_vars["SupplierAddress"].set(supplier.get("Address", "") or "")
            self.proc_vars["SupplierABN"].set(supplier.get("ABN", "") or "")
            self.proc_vars["SupplierPaymentTerms"].set(supplier.get("PaymentTerms", "") or "")

    self.contact_id_map = {}
    values = []
    for c in contacts:
        label = f"{c.get('ContactName', '')} ({c.get('Role', '')})".strip()
        values.append(label)
        self.contact_id_map[label] = c.get("ContactID")
    self.contact_combo["values"] = values

    primary = next((c for c in contacts if str(c.get("IsPrimary", "")).lower() in ("true", "yes", "1")), None)
    if primary:
        label = f"{primary.get('ContactName', '')} ({primary.get('Role', '')})".strip()
        self.proc_vars["ContactID"].set(label)
        self._on_contact_changed()
    elif values:
        self.proc_vars["ContactID"].set("")
        self._clear_contact_details()
    else:
        self.proc_vars["ContactID"].set("")

```

```

        self._clear_contact_details()
except Exception as error:
    messagebox.showerror("Supplier Load Error", str(error), parent=self.root)

def _on_contact_changed(self, event=None):
    label = self.proc_vars["ContactID"].get()
    contact_id = self.contact_id_map.get(label)
    if not contact_id:
        self._clear_contact_details()
        return
    try:
        wb = load_workbook(self.excel_filepath)
        contacts = get_all_supplier_contacts(wb,
self.supplier_id_map.get(self.proc_vars["SupplierID"].get()))
        wb.close()
        contact = next((c for c in contacts if str(c.get("ContactID")) == str(contact_id)), None)
        if contact:
            self.proc_vars["ContactRole"].set(contact.get("Role", "") or "")
            self.proc_vars["ContactPhone"].set(contact.get("PhoneNumber", "") or "")
            self.proc_vars["ContactMobile"].set(contact.get("MobileNumber", "") or "")
            self.proc_vars["ContactEmail"].set(contact.get("Email", "") or "")
        except Exception as error:
            messagebox.showerror("Contact Load Error", str(error), parent=self.root)

def _clear_contact_details(self):
    for key in ["ContactRole", "ContactPhone", "ContactMobile", "ContactEmail"]:
        self.proc_vars[key].set("")

def _on_agreement_type_changed(self, event=None):
    labels = {
        "Block": "Block details",
        "Coupe": "Coupe details",
        "Harvest": "Harvest details",
        "Contract Number": "Contract number",
    }
    self.agreement_detail_label.configure(
        text=labels.get(self.proc_vars["AgreementType"].get(), "Details")
    )

# ----- Grade table -----

def _load_grade_settings(self):
    """Load the shared species/product/grade choices for new rows."""
    try:
        saved_species = get_setting("grade_table_species", "")
        saved_product = get_setting("grade_table_product_type", "Fresh Logs") or "Fresh Logs"
        saved_grades = get_setting("grade_table_grades", "")

        wb = load_workbook(self.excel_filepath)
        species_records = get_species_definitions(wb)
        species_names = [r.get("SpeciesName", "") for r in species_records if r.get("SpeciesName")]
        if saved_species and saved_species not in species_names:

```

```

        add_species_definition(wb, saved_species)
        save_workbook(wb, self.excel_filepath)
        species_names.append(saved_species)
        definitions = get_grade_definitions(wb, saved_species, saved_product) if saved_species else
get_grade_definitions(wb, "", saved_product)
        wb.close()

```

```

grades = [g.strip() for g in saved_grades.split("|") if g.strip()]
if not grades:
    grades = [r.get("GradeName", "") for r in definitions if r.get("GradeName")]

```

```

self._grade_row_settings = {
    "species": saved_species,
    "product_type": saved_product,
    "grades": grades,
}

```

except Exception:

```

self._grade_row_settings = {
    "species": "",
    "product_type": "Fresh Logs",
    "grades": [],
}

```

def _open_grade_settings(self):

"""Manage the shared species/product grade choices used by procurement rows."""

```

dialog = tk.Toplevel(self.root)
dialog.title("Grade Table Settings")
saved_geometry = get_setting("grade_settings_geometry", "")
try:

```

```

    if saved_geometry:
        dialog.geometry(saved_geometry)
    else:
        dialog.geometry("520x560")

```

```

except tk.TclError:
    dialog.geometry("520x560")
dialog.minsize(480, 480)
dialog.transient(self.root)
dialog.grab_set()

```

```

frame = ttk.Frame(dialog, padding=18)
frame.pack(fill="both", expand=True)
frame.columnconfigure(1, weight=1)
frame.rowconfigure(4, weight=1)

```

```

ttk.Label(
    frame, text="Grade Table Settings",
    font=("Segoe UI", 14, "bold")
).grid(row=0, column=0, columnspan=2, sticky="w", pady=(0, 15))

```

```

ttk.Label(frame, text="Species").grid(row=1, column=0, sticky="w", padx=5, pady=6)
species_var = tk.StringVar(value=self._grade_row_settings.get("species", ""))
species_combo = ttk.Combobox(frame, textvariable=species_var, state="readonly")

```

```

species_combo.grid(row=1, column=1, sticky="ew", padx=5, pady=6)

ttk.Label(frame, text="Product Type").grid(row=2, column=0, sticky="w", padx=5, pady=6)
product_var = tk.StringVar(value=self._grade_row_settings.get("product_type", "Fresh Logs"))
product_combo = ttk.Combobox(
    frame, textvariable=product_var,
    values=PRODUCT_TYPES, state="readonly"
)
product_combo.grid(row=2, column=1, sticky="ew", padx=5, pady=6)

species_add_frame = ttk.Frame(frame)
species_add_frame.grid(row=3, column=1, sticky="ew", padx=5, pady=(0, 8))
species_add_frame.columnconfigure(0, weight=1)
new_species_var = tk.StringVar()
ttk.Entry(species_add_frame, textvariable=new_species_var).grid(row=0, column=0, sticky="ew")
ttk.Button(
    species_add_frame, text="Add Species", command=lambda: add_species(), width=12
).grid(row=0, column=1, padx=(6, 0))

grades_label = ttk.Label(frame, text="Available Grades")
grades_label.grid(row=4, column=0, sticky="nw", padx=5, pady=6)

grades_frame = ttk.Frame(frame)
grades_frame.grid(row=4, column=1, sticky="nsew", padx=5, pady=6)
grades_frame.rowconfigure(0, weight=1)
grades_frame.columnconfigure(0, weight=1)

grades_list = tk.Listbox(grades_frame, selectmode="browse", height=12)
grades_list.grid(row=0, column=0, sticky="nsew")
grades_scroll = ttk.Scrollbar(grades_frame, orient="vertical", command=grades_list.yview)
grades_scroll.grid(row=0, column=1, sticky="ns")
grades_list.config(yscrollcommand=grades_scroll.set)

new_grade_frame = ttk.Frame(frame)
new_grade_frame.grid(row=5, column=1, sticky="ew", padx=5, pady=8)
new_grade_frame.columnconfigure(0, weight=1)
new_grade_var = tk.StringVar()
new_grade_entry = ttk.Entry(new_grade_frame, textvariable=new_grade_var)
new_grade_entry.grid(row=0, column=0, sticky="ew")

grade_action_frame = ttk.Frame(frame)
grade_action_frame.grid(row=6, column=1, sticky="e", padx=5, pady=(0, 8))

grade_definition_ids = []

def clean_grade_name(name):
    name = str(name or "").strip()
    if name.lower().endswith(" grade"):
        return name[:-6].strip()
    return name

def load_species_choices():

```

```

try:
    wb = load_workbook(self.excel_filepath)
    names = [r.get("SpeciesName", "") for r in get_species_definitions(wb) if r.get("SpeciesName")]
    wb.close()
    species_combo["values"] = names
except Exception:
    species_combo["values"] = []

```

```

def load_grades_for_selection(event=None):
    nonlocal grade_definition_ids
    try:
        wb = load_workbook(self.excel_filepath)
        definitions = get_grade_definitions(wb, species_var.get(), product_var.get())
        wb.close()
        grade_definition_ids = [r.get("GradeDefinitionID") for r in definitions if r.get("GradeName")]
        grades_list.delete(0, tk.END)
        for record in definitions:
            if record.get("GradeName"):
                grades_list.insert(tk.END, clean_grade_name(record.get("GradeName")))

        # Preserve the user's configured list for the currently configured
        # species/product combination.
        saved = self._grade_row_settings.get("grades", [])
        if (species_var.get() == self._grade_row_settings.get("species") and
            product_var.get() == self._grade_row_settings.get("product_type") and saved):
            grades_list.delete(0, tk.END)
            for name in saved:
                grades_list.insert(tk.END, clean_grade_name(name))
        # IDs are rebuilt by matching the displayed names.
        try:
            wb = load_workbook(self.excel_filepath)
            definitions = get_grade_definitions(wb, species_var.get(), product_var.get())
            wb.close()
            grade_definition_ids = []
            for display_name in saved:
                target = clean_grade_name(display_name).lower()
                match = next((r for r in definitions
                             if clean_grade_name(r.get("GradeName")).lower() == target), None)
                grade_definition_ids.append(match.get("GradeDefinitionID") if match else None)
        except Exception:
            grade_definition_ids = [None] * len(saved)
    except Exception:
        grade_definition_ids = []
        grades_list.delete(0, tk.END)

```

```

def add_species():
    name = new_species_var.get().strip()
    if not name:
        messagebox.showerror("Validation Error", "Please enter a species name.", parent=dialog)
        return
    try:
        wb = load_workbook(self.excel_filepath)

```

```

existing = [r.get("SpeciesName", "") for r in get_species_definitions(wb)]
if any(name.lower() == str(x).strip().lower() for x in existing):
    messagebox.showwarning("Species Exists", "That species already exists.", parent=dialog)
    wb.close()
    return
add_species_definition(wb, name)
save_workbook(wb, self.excel_filepath)
wb.close()
load_species_choices()
species_var.set(name)
new_species_var.set("")
load_grades_for_selection()
except Exception as error:
    messagebox.showerror("Save Error", str(error), parent=dialog)

def add_new_grade():
    name = new_grade_var.get().strip()
    species = species_var.get().strip()
    product = product_var.get().strip()
    if not name:
        return
    if not species:
        messagebox.showerror("Validation Error", "Select or add a species first.", parent=dialog)
        return
    if product not in PRODUCT_TYPES:
        messagebox.showerror("Validation Error", "Please select a product type.", parent=dialog)
        return
    try:
        wb = load_workbook(self.excel_filepath)
        add_grade_definition(wb, species, product, clean_grade_name(name))
        save_workbook(wb, self.excel_filepath)
        wb.close()
        load_grades_for_selection()
        new_grade_var.set("")
        new_grade_entry.focus_set()
    except Exception as error:
        messagebox.showerror("Save Error", str(error), parent=dialog)

def edit_selected_grade():
    selection = grades_list.curselection()
    if not selection:
        messagebox.showwarning("Select Grade", "Select a grade to edit.", parent=dialog)
        return
    index = selection[0]
    grade_id = grade_definition_ids[index] if index < len(grade_definition_ids) else None
    old_display = grades_list.get(index)
    if not grade_id:
        messagebox.showerror("Edit Error", "The selected grade could not be identified.",
parent=dialog)
        return
    new_name = simpdialog.askstring(
        "Edit Grade", "Grade name:", initialValue=old_display, parent=dialog

```

```

)
if new_name is None:
    return
new_name = clean_grade_name(new_name)
if not new_name:
    messagebox.showerror("Validation Error", "Grade name is required.", parent=dialog)
    return
try:
    wb = load_workbook(self.excel_filepath)
    update_grade_definition(wb, grade_id, {"GradeName": new_name})
    save_workbook(wb, self.excel_filepath)
    wb.close()
    load_grades_for_selection()
except Exception as error:
    messagebox.showerror("Edit Error", str(error), parent=dialog)

def remove_selected_grade():
    selection = grades_list.curselection()
    if not selection:
        messagebox.showwarning("Select Grade", "Select a grade to remove.", parent=dialog)
        return
    index = selection[0]
    grade_id = grade_definition_ids[index] if index < len(grade_definition_ids) else None
    grade_name = grades_list.get(index)
    if not grade_id:
        messagebox.showerror("Remove Error", "The selected grade could not be identified.",
parent=dialog)
        return
    if not messagebox.askyesno(
        "Remove Grade", f"Remove grade '{grade_name}' from this species/product type?",
parent=dialog
    ):
        return
    try:
        wb = load_workbook(self.excel_filepath)
        delete_grade_definition(wb, grade_id)
        save_workbook(wb, self.excel_filepath)
        wb.close()
        load_grades_for_selection()
    except Exception as error:
        messagebox.showerror("Remove Error", str(error), parent=dialog)

ttk.Button(new_grade_frame, text="Add Grade", command=add_new_grade, width=12).grid(
    row=0, column=1, padx=(6, 0)
)
ttk.Button(grade_action_frame, text="Edit Grade", command=edit_selected_grade,
width=12).pack(side="left", padx=3)
ttk.Button(grade_action_frame, text="Remove Grade", command=remove_selected_grade,
width=12).pack(side="left", padx=3)

def save_settings():
    species = species_var.get().strip()

```

```

product = product_var.get().strip()
grades = [grades_list.get(i).strip() for i in range(grades_list.size()) if grades_list.get(i).strip()]

if not species:
    messagebox.showerror("Validation Error", "Please select or add a species.", parent=dialog)
    return
if product not in PRODUCT_TYPES:
    messagebox.showerror("Validation Error", "Please select a product type.", parent=dialog)
    return
if not grades:
    messagebox.showerror("Validation Error", "Please select or add at least one grade.",
parent=dialog)
    return

self._grade_row_settings = {
    "species": species,
    "product_type": product,
    "grades": grades,
}
set_setting("grade_table_species", species)
set_setting("grade_table_product_type", product)
set_setting("grade_table_grades", "|".join(grades))

# Refresh all existing rows with the currently configured choices.
species_values = list(species_combo["values"])
for row in self.grade_rows:
    row["species_combo"]["values"] = species_values
    row["product_combo"]["values"] = PRODUCT_TYPES
    row["grade_combo"]["values"] = self._get_row_grades(
        row["species_var"].get(), row["product_var"].get()
    )

_save_dialog_geometry()
dialog.destroy()

def _save_dialog_geometry():
    try:
        dialog.update_idletasks()
        set_setting("grade_settings_geometry", dialog.geometry())
    except Exception:
        pass

def close_dialog():
    _save_dialog_geometry()
    dialog.destroy()

buttons = ttk.Frame(frame)
buttons.grid(row=7, column=0, columnspan=2, sticky="ew", pady=(12, 0))
ttk.Button(buttons, text="Save", command=save_settings, width=18).pack(side="left", padx=5)
ttk.Button(buttons, text="Cancel", command=close_dialog, width=18).pack(side="left", padx=5)
dialog.protocol("WM_DELETE_WINDOW", close_dialog)

```

```

load_species_choices()
if species_var.get() and species_var.get() not in species_combo["values"]:
    species_var.set("")
if species_var.get():
    load_grades_for_selection()
else:
    species_values = list(species_combo["values"])
    if species_values:
        species_var.set(species_values[0])
    load_grades_for_selection()

def _get_row_grades(self, species, product_type):
    """Return grade names available for a species/product row."""
    try:
        wb = load_workbook(self.excel_filepath)
        definitions = get_grade_definitions(wb, species, product_type)
        wb.close()
        names = []
        for record in definitions:
            name = str(record.get("GradeName", "")).strip()
            if name:
                if name.lower().endswith(" grade"):
                    name = name[:-6].strip()
                names.append(name)
        # For the currently configured species/product, preserve the user's
        # configured grade list if it exists.
        if (species == self._grade_row_settings.get("species") and
            product_type == self._grade_row_settings.get("product_type") and
            self._grade_row_settings.get("grades")):
            return [str(x).strip() for x in self._grade_row_settings["grades"] if str(x).strip()]
        return names
    except Exception:
        return []

def _add_grade_row(self, focus_grade=True):
    """Add an editable species/product/grade row directly into the grade table."""
    row_index = len(self.grade_rows) + 1
    settings = self._grade_row_settings

    species_values = []
    try:
        wb = load_workbook(self.excel_filepath)
        species_values = [r.get("SpeciesName", "") for r in get_species_definitions(wb) if
r.get("SpeciesName")]
        wb.close()
    except Exception:
        species_values = []

    species_default = settings.get("species", "") or (species_values[0] if species_values else "")
    product_default = settings.get("product_type", "Fresh Logs") or "Fresh Logs"

    species_var = tk.StringVar(value=species_default)

```

```

product_var = tk.StringVar(value=product_default)
grade_var = tk.StringVar()
offered_var = tk.StringVar()
agreed_price_var = tk.StringVar()
agreed_tonnes_var = tk.StringVar()
delivered_var = tk.StringVar(value="0")
remaining_var = tk.StringVar(value="0.00")

species_combo = ttk.Combobox(
    self.grade_table_frame, textvariable=species_var,
    values=species_values, state="readonly"
)
product_combo = ttk.Combobox(
    self.grade_table_frame, textvariable=product_var,
    values=PRODUCT_TYPES, state="readonly"
)
grade_combo = ttk.Combobox(
    self.grade_table_frame, textvariable=grade_var,
    values=self._get_row_grades(species_default, product_default), state="readonly"
)
offered_entry = ttk.Entry(self.grade_table_frame, textvariable=offered_var)
agreed_price_entry = ttk.Entry(self.grade_table_frame, textvariable=agreed_price_var)
agreed_tonnes_entry = ttk.Entry(self.grade_table_frame, textvariable=agreed_tonnes_var)
delivered_entry = ttk.Entry(self.grade_table_frame, textvariable=delivered_var)
remaining_entry = ttk.Entry(
    self.grade_table_frame, textvariable=remaining_var, state="readonly"
)

widgets = [
    species_combo, product_combo, grade_combo, offered_entry,
    agreed_price_entry, agreed_tonnes_entry, delivered_entry, remaining_entry
]
for col, widget in enumerate(widgets):
    widget.grid(row=row_index, column=col, sticky="ew", padx=2, pady=2)

def refresh_grade_choices(event=None):
    grade_values = self._get_row_grades(species_var.get(), product_var.get())
    grade_combo["values"] = grade_values
    if grade_var.get() not in grade_values:
        grade_var.set("")

species_combo.bind("<<ComboboxSelected>>", refresh_grade_choices)
product_combo.bind("<<ComboboxSelected>>", refresh_grade_choices)

def update_remaining(*_):
    try:
        agreed = float(agreed_tonnes_var.get() or 0)
        delivered = float(delivered_var.get() or 0)
        remaining_var.set(f"{max(agreed - delivered, 0):.2f}")
    except ValueError:
        remaining_var.set("0.00")

```

```

agreed_tonnes_var.trace_add("write", update_remaining)
delivered_var.trace_add("write", update_remaining)

row_data = {
    "species_var": species_var,
    "species_combo": species_combo,
    "product_var": product_var,
    "product_combo": product_combo,
    "grade_var": grade_var,
    "grade_combo": grade_combo,
    "offered_var": offered_var,
    "agreed_price_var": agreed_price_var,
    "agreed_tonnes_var": agreed_tonnes_var,
    "delivered_var": delivered_var,
    "remaining_var": remaining_var,
    "widgets": widgets,
}
self.grade_rows.append(row_data)
self._move_grade_table_bottom_border()

if focus_grade:
    species_combo.focus_set()

def _move_grade_table_bottom_border(self):
    """Keep a visible white row below the grade list as a clean lower border."""
    if not hasattr(self, "grade_table_bottom_border"):
        return
    border_row = len(self.grade_rows) + 1
    self.grade_table_bottom_border.grid_configure(row=border_row, column=0, columnspan=8,
sticky="ew")

def _remove_grade_row(self):
    if not self.grade_rows:
        return
    # Remove the last row unless the user has selected an entry widget.
    selected = self.root.focus_get()
    selected_index = None
    for index, row in enumerate(self.grade_rows):
        if selected in row["widgets"]:
            selected_index = index
            break

    if selected_index is None:
        selected_index = len(self.grade_rows) - 1

    row = self.grade_rows.pop(selected_index)
    for widget in row["widgets"]:
        widget.destroy()

    # Re-grid remaining rows so Tab order remains clean.
    for index, row in enumerate(self.grade_rows, start=1):
        for col, widget in enumerate(row["widgets"]):

```

```

        widget.grid_configure(row=index, column=col)
self._move_grade_table_bottom_border()

def _clear_grade_rows(self):
    for row in self.grade_rows:
        for widget in row["widgets"]:
            widget.destroy()
    self.grade_rows = []
    self._move_grade_table_bottom_border()

@staticmethod
def _remaining(agreed, delivered):
    try:
        return f"{max(float(agreed or 0) - float(delivered or 0), 0):.2f}"
    except (ValueError, TypeError):
        return "0.00"

def _grade_rows(self):
    rows = []
    for row in self.grade_rows:
        species = row["species_var"].get().strip()
        product_type = row["product_var"].get().strip()
        grade = row["grade_var"].get().strip()
        offered_text = row["offered_var"].get().strip()
        agreed_price_text = row["agreed_price_var"].get().strip()
        agreed_tonnes_text = row["agreed_tonnes_var"].get().strip()
        delivered_text = row["delivered_var"].get().strip()

        if not grade and not any([
            offered_text, agreed_price_text, agreed_tonnes_text, delivered_text
        ]):
            continue

        if not species:
            raise ValueError("Set a species using the ⌂ button before saving.")
        if not product_type:
            raise ValueError("Set a product type using the ⌂ button before saving.")
        if not grade:
            raise ValueError("Please select a grade for every entered row.")

    try:
        offered = float(offered_text or 0)
        agreed_price = float(agreed_price_text or 0)
        agreed_tonnes = float(agreed_tonnes_text or 0)
        delivered = float(delivered_text or 0)
    except ValueError:
        raise ValueError(f"Prices and tonnes must be numbers for grade {grade}.")

    if offered < 0:
        raise ValueError(f"Offered price cannot be negative for {grade}.")
    if agreed_price <= 0:
        raise ValueError(f"Agreed price must be greater than 0 for {grade}.")

```

```

if agreed_tonnes <= 0:
    raise ValueError(f"Agreed tonnes must be greater than 0 for {grade}.")
if delivered < 0 or delivered > agreed_tonnes:
    raise ValueError(f"Delivered tonnes must be between 0 and Agreed Tonnes for {grade}.")

```

```

rows.append({
    "Species": species,
    "ProductType": product_type,
    "GradeName": grade,
    "OfferedPricePerTonne": offered,
    "AgreedPricePerTonne": agreed_price,
    "AgreedTonnes": agreed_tonnes,
    "DeliveredTonnes": delivered,
    "RemainingTonnes": max(agreed_tonnes - delivered, 0),
    "Notes": "",
})

```

```

if not rows:
    raise ValueError("Add at least one grade row before saving the procurement.")
return rows

```

```

def _form_data(self):
    data = {key: var.get().strip() for key, var in self.proc_vars.items()}
    if key not in {"SupplierAddress", "SupplierABN", "SupplierPaymentTerms",
                  "ContactRole", "ContactPhone", "ContactMobile", "ContactEmail"}}
    data["SupplierID"] = self.supplier_id_map.get(data.get("SupplierID"), data.get("SupplierID", ""))
    data["ContactID"] = self.contact_id_map.get(data.get("ContactID"), data.get("ContactID", ""))
    data["AcceptanceNotes"] = self.proc_acceptance_notes.get("1.0", tk.END).strip()
    data["Notes"] = self.proc_notes.get("1.0", tk.END).strip()
    data["ChangedBy"] = "Current User"
    data["ChangedDate"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    data["CreatedBy"] = "Current User"
    data["CreatedDate"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return data

```

```

def _validate_procurement(self, data):
    if not data.get("SupplierID"):
        raise ValueError("Please select a supplier.")
    if not data.get("ContactID"):
        raise ValueError("Please select a supplier contact.")
    if not data.get("AgreementType"):
        raise ValueError("Please select an Agreement Type.")
    if not data.get("AgreementDetail"):
        raise ValueError("Please enter the selected agreement detail.")
    if not data.get("Status") in STATUSES:
        raise ValueError("Please select a valid status.")
    if not data.get("HarvestPeriodStart") or not data.get("HarvestPeriodEnd"):
        raise ValueError("Please enter both Harvest Period Start and Harvest Period End.")
    self._grade_rows()

```

```

def _save_procurement(self):
    try:

```

```

data = self._form_data()
self._validate_procurement(data)
grades = self._grade_rows()
wb = load_workbook(self.excel_filepath)
_, ref = add_procurement(wb, data)
for grade in grades:
    grade["ProcurementRef"] = ref
    add_procurement_grade(wb, grade)
save_workbook(wb, self.excel_filepath)
wb.close()
self.current_procurement_ref = ref
self._refresh_procurement_table()
self._refresh_home()
messagebox.showinfo("Saved", f"Procurement {ref} saved successfully.", parent=self.root)
except Exception as error:
    messagebox.showerror("Save Error", str(error), parent=self.root)

def _update_procurement(self):
    if not self.current_procurement_ref:
        messagebox.showwarning("No Procurement Selected",
                                "Select a procurement from the register first.", parent=self.root)
    return
    # Full grade-level edit/history logic is intentionally the next refinement;
    # this chapter focuses on getting the new data-entry layout working safely.
    messagebox.showinfo(
        "Chapter 3.1",
        "The new procurement form is ready. Full grade-level editing and price-history update logic will
be completed in the next procurement step.",
        parent=self.root,
    )

def _on_procurement_selected(self, event=None):
    selection = self.proc_tree.selection()
    if not selection:
        return
    ref = self.proc_tree.item(selection[0], "values")[0]
    try:
        wb = load_workbook(self.excel_filepath)
        record = get_procurement_by_ref(wb, ref)
        suppliers = get_all_suppliers(wb)
        contacts = get_all_supplier_contacts(wb, record.get("SupplierID")) if record else []
        grades = get_all_procurement_grades(wb, ref)
        wb.close()
        if not record:
            return
        self.current_procurement_ref = ref

        supplier_display = ""
        for s in suppliers:
            if str(s.get("SupplierID")) == str(record.get("SupplierID")):
                supplier_display = f"{s.get('SupplierName', '')} (ID: {s.get('SupplierID', '')})"
                break

```

```
self.proc_vars["SupplierID"].set(supplier_display)
self._on_supplier_changed()
```

```
for c in contacts:
    if str(c.get("ContactID")) == str(record.get("ContactID")):
        label = f'{c.get("ContactName", "")} {c.get("Role", "")}'.strip()
        self.proc_vars["ContactID"].set(label)
        self._on_contact_changed()
        break
```

```
for key, var in self.proc_vars.items():
    if key in {"SupplierID", "ContactID", "SupplierAddress", "SupplierABN",
              "SupplierPaymentTerms", "ContactRole", "ContactPhone",
              "ContactMobile", "ContactEmail"}:
        continue
    var.set(str(record.get(key, "") or ""))
```

```
self.proc_acceptance_notes.delete("1.0", tk.END)
self.proc_acceptance_notes.insert("1.0", record.get("AcceptanceNotes", "") or "")
self.proc_notes.delete("1.0", tk.END)
self.proc_notes.insert("1.0", record.get("Notes", "") or "")
```

```
self._clear_grade_rows()
for g in grades:
    self._add_grade_row(focus_grade=False)
    row = self.grade_rows[-1]
    row["species_var"].set(g.get("Species", "") or "")
    row["product_var"].set(g.get("ProductType", "") or "")
    row["grade_combo"]["values"] = self._grade_row_settings.get("grades", [])
    row["grade_var"].set(g.get("GradeName", "") or "")
    row["offered_var"].set(str(g.get("OfferedPricePerTonne", "") or ""))
    row["agreed_price_var"].set(str(g.get("AgreedPricePerTonne", "") or ""))
    row["agreed_tonnes_var"].set(str(g.get("AgreedTonnes", "") or ""))
    row["delivered_var"].set(str(g.get("DeliveredTonnes", "") or ""))
```

```
except Exception as error:
    messagebox.showerror("Load Error", str(error), parent=self.root)
```

```
def _clear_procurement_form(self):
    self.current_procurement_ref = None
    for key, var in self.proc_vars.items():
        if key.startswith("Supplier") or key.startswith("Contact"):
            var.set("")
        else:
            var.set("")
    self.proc_vars["Status"].set("Draft")
    self.proc_acceptance_notes.delete("1.0", tk.END)
    self.proc_notes.delete("1.0", tk.END)
    self._clear_grade_rows()
    self._add_grade_row(focus_grade=False)
    self.contact_combo["values"] = []
```

```
def _refresh_procurement_table(self):
```

```

if not hasattr(self, "proc_tree"):
    return
for item in self.proc_tree.get_children():
    self.proc_tree.delete(item)
try:
    wb = load_workbook(self.excel_filepath)
    records = get_all_procurements(wb)
    suppliers = get_all_suppliers(wb)
    grades = get_all_procurement_grades(wb)
    wb.close()
    supplier_lookup = {str(s["SupplierID"]): s["SupplierName"] for s in suppliers}
    totals = {}
    for g in grades:
        ref = g.get("ProcurementRef")
        totals.setdefault(ref, [0.0, 0.0])
        totals[ref][0] += float(g.get("AgreedTonnes") or 0)
        totals[ref][1] += float(g.get("DeliveredTonnes") or 0)

    for r in records:
        supplier = supplier_lookup.get(str(r.get("SupplierID")), "")
        agreed, delivered = totals.get(r.get("ProcurementRef"), (0, 0))
        self.proc_tree.insert("", "end", values=(
            r.get("ProcurementRef", ""), supplier, r.get("AgreementType", ""),
            r.get("AgreementDetail", ""), r.get("Plantation", ""),
            r.get("Species", ""), f"{agreed:,.2f}", f"{delivered:,.2f}",
            f"{max(agreed-delivered, 0):,.2f}", r.get("Status", "")
        ))
    self._refresh_home()
except Exception as error:
    messagebox.showerror("Load Error", f"Could not load procurements:\n{error}", parent=self.root)

```

----- Price History -----

```

def _init_price_history_tab(self):
    frame = ttk.Frame(self.tabs["Price History"], padding=15)
    frame.pack(fill="both", expand=True)
    ttk.Label(frame, text="Price History",
               font=("Segoe UI", 16, "bold")).pack(anchor="w", pady=(0, 10))
    columns = ("Ref", "ProductType", "Grade", "Previous", "New", "Reason", "Effective", "User")
    self.price_tree = ttk.Treeview(frame, columns=columns, show="headings")
    headings = {"Ref": "Procurement", "ProductType": "Product Type", "Grade": "Grade",
                "Previous": "Previous Price", "New": "New Price", "Reason": "Reason",
                "Effective": "Effective Date/Time", "User": "Recorded By"}
    for col in columns:
        self.price_tree.heading(col, text=headings[col])
        self.price_tree.column(col, width=150, minwidth=80)
    self.price_tree.pack(fill="both", expand=True)

def _refresh_price_history(self):
    if not hasattr(self, "price_tree"):
        return
    for item in self.price_tree.get_children():

```

```

        self.price_tree.delete(item)
try:
    wb = load_workbook(self.excel_filepath)
    records = get_all_price_history(wb)
    wb.close()
    for r in records:
        self.price_tree.insert("", "end", values=(
            r.get("ProcurementRef", ""), r.get("ProductType", ""),
            r.get("GradeName", ""), r.get("PreviousPrice", ""),
            r.get("NewPrice", ""), r.get("Reason", ""),
            r.get("EffectiveDateTime", ""), r.get("RecordedBy", "")
        ))
except Exception as error:
    messagebox.showerror("Load Error", str(error), parent=self.root)

# ----- Suppliers -----

def _init_suppliers_tab(self):
    tab = self.tabs["Suppliers"]
    frame = ttk.Frame(tab, padding=15)
    frame.pack(fill="both", expand=True)
    ttk.Label(frame, text="Suppliers",
        font=("Segoe UI", 16, "bold")).pack(anchor="w")
    button_frame = ttk.Frame(frame)
    button_frame.pack(anchor="w", pady=10)
    ttk.Button(button_frame, text="Add Supplier",
        command=self._add_supplier, width=18).pack(side="left", padx=(0, 6))
    ttk.Button(button_frame, text="Open Selected Supplier",
        command=self._open_selected_supplier, width=22).pack(side="left", padx=6)
    ttk.Button(button_frame, text="Refresh",
        command=self._refresh_suppliers, width=12).pack(side="left", padx=6)

    columns = ("ID", "Name", "Address", "ABN", "Terms", "Contacts", "Notes")
    self.suppliers_tree = ttk.Treeview(frame, columns=columns, show="headings")
    headings = {"ID": "ID", "Name": "Supplier Name", "Address": "Address",
        "ABN": "ABN", "Terms": "Payment Terms", "Contacts": "Contacts",
        "Notes": "Notes"}
    for col in columns:
        self.suppliers_tree.heading(col, text=headings[col])
        self.suppliers_tree.column(col, width=180, minwidth=80)
    self.suppliers_tree.pack(fill="both", expand=True)
    self.suppliers_tree.bind("<Double-1>", lambda event: self._open_selected_supplier())

def _open_selected_supplier(self):
    selection = self.suppliers_tree.selection()
    if not selection:
        messagebox.showwarning("Select Supplier", "Please select a supplier first.", parent=self.root)
        return
    supplier_id = self.suppliers_tree.item(selection[0], "values")[0]
    dialog = ManageSupplierDialog(self.root, self.excel_filepath, supplier_id)
    if dialog.result:
        self._refresh_suppliers()

```

```

self._refresh_supplier_choices()
self._on_supplier_changed()

def _add_supplier(self):
    dialog = AddSupplierDialog(self.root, self.excel_filepath)
    if dialog.result:
        self._refresh_suppliers()
        self._refresh_supplier_choices()

        # Make the newly saved supplier immediately available and selected
        # on the Procurement Agreement page.
        if dialog.result_supplier_id is not None:
            for display, supplier_id in self.supplier_id_map.items():
                if str(supplier_id) == str(dialog.result_supplier_id):
                    self.proc_vars["SupplierID"].set(display)
                    self._on_supplier_changed()
                    break

        messagebox.showinfo("Supplier Saved",
                            "The supplier and its contacts have been saved.\n\nThe Procurement supplier list has
                            been refreshed.", parent=self.root)

def _refresh_suppliers(self):
    if not hasattr(self, "suppliers_tree"):
        return
    for item in self.suppliers_tree.get_children():
        self.suppliers_tree.delete(item)
    try:
        wb = load_workbook(self.excel_filepath)
        suppliers = get_all_suppliers(wb)
        wb.close()
        for s in suppliers:
            wb2 = load_workbook(self.excel_filepath)
            contacts = get_all_supplier_contacts(wb2, s.get("SupplierID"))
            wb2.close()
            contact_names = ", ".join(str(c.get("ContactName", "")) for c in contacts)
            self.suppliers_tree.insert("", "end", values=(
                s.get("SupplierID", ""), s.get("SupplierName", ""),
                s.get("Address", ""), s.get("ABN", ""),
                s.get("PaymentTerms", ""), contact_names, s.get("Notes", "")
            ))
    except Exception as error:
        messagebox.showerror("Load Error", str(error), parent=self.root)

def _on_close(self):
    """Save the current window size and position before closing."""
    try:
        self.root.update_idletasks()
        geometry = self.root.geometry()
        set_setting("window_geometry", geometry)
    except Exception:
        pass

```

```
self.root.destroy()
```

```
def _init_placeholder_tabs(self):  
    for name in ["Costing", "Reports", "Resales", "Settings"]:  
        ttk.Label(self.tabs[name], text=f"{name} - Coming later",  
                  font=("Segoe UI", 12)).pack(pady=40)
```

main.py

```
# main.py
```

```
# Entry point for the Log Procurement Tracker app.
```

```
import tkinter as tk
```

```
import os
```

```
from gui_layer import MainWindow, FirstRunDialog
```

```
from utils import get_setting
```

```
def main():
```

```
    # Check if we have a configured Excel file path
```

```
    excel_path = get_setting("last_file_path", "")
```

```
    # If no path configured, or file doesn't exist, show first-run dialog
```

```
    if not excel_path or not os.path.exists(excel_path):
```

```
        # Show first-run dialog (it has its own root window)
```

```
        dialog = FirstRunDialog()
```

```
        excel_path = dialog.result_path
```

```
    # If user cancelled, exit app
```

```
    if not excel_path:
```

```
        print("No file selected. Exiting.")
```

```
        return
```

```
    # Create main application window
```

```
root = tk.Tk()

app = MainWindow(root, excel_path)


# Start the GUI event loop

root.mainloop()


if __name__ == "__main__":

    main()
```

requirement.txt

```
openpyxl
pyinstaller
pycryptodome
```

utils.py

```
# utils.py
# Helper functions for configuration and common utilities.


import os


CONFIG_FILE = "config.txt"


def get_config_path():
    """
    Return the full path to config.txt (in the same folder as main.py).
    """
    script_dir = os.path.dirname(os.path.abspath(__file__))
    return os.path.join(script_dir, CONFIG_FILE)


def load_config():
    """
    Load configuration from config.txt.
    Returns a dictionary of key-value pairs.

    File format (one per line):
    key=value

    Lines starting with # are comments.
    """
    config = {}
```

```

config_path = get_config_path()

if not os.path.exists(config_path):
    # Return defaults if file doesn't exist
    return {
        "excel_password": "",
        "last_file_path": "",
        "login_required": "False",
    }

with open(config_path, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        if "=" in line:
            key, value = line.split("=", 1)
            config[key.strip()] = value.strip()

# Ensure required keys exist
defaults = {
    "excel_password": "",
    "last_file_path": "",
    "login_required": "False",
}
for key, default_value in defaults.items():
    if key not in config:
        config[key] = default_value

return config

def save_config(config_dict):
    """
    Save configuration dictionary to config.txt.
    Overwrites existing file.
    """
    config_path = get_config_path()

    with open(config_path, "w", encoding="utf-8") as f:
        f.write("# Log Procurement Tracker Configuration\n")
        f.write("# Do not edit manually unless you know what you're doing.\n\n")
        for key, value in config_dict.items():
            f.write(f"{key}={value}\n")

def get_setting(key, default=""):
    """
    Get a single setting value by key.
    Returns default if key doesn't exist.
    """
    config = load_config()
    return config.get(key, default)

```

```
def set_setting(key, value):  
    """  
    Set a single setting value and save to config.txt.  
    """  
    config = load_config()  
    config[key] = value  
    save_config(config)
```